C Programming Language

A Comprehensive Guide for Programmers

Syntax, Semantics, and Patterns

1. Introduction & Overview

C is a general-purpose, procedural, compiled programming language developed at Bell Labs by Dennis Ritchie in 1972. It is the grandfather of most modern languages — Python, Java, C++, JavaScript, and Swift all inherit syntax and concepts from C. If you already know any of these, C will feel both familiar and refreshingly low-level.

Key Characteristics

- Compiled — source code is translated directly to machine code by a compiler (gcc, clang)
- Statically typed — all variable types are declared at compile time
- Manual memory management — you allocate and free memory yourself (malloc/free)
- Procedural — code is organized into functions, not objects or classes
- Low-level access — direct pointer arithmetic and bit manipulation
- Portable — write once, compile anywhere (mostly)

Hello World — Your First C Program

```
#include <stdio.h>


int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Compile and run: `gcc hello.c -o hello && ./hello`

Element	Meaning
`#include <stdio.h>`	Import the standard I/O library header
`int main(void)`	Entry point — every C program starts here. Returns int.

Element	Meaning
`printf(...)`	Print formatted output to stdout
`return 0;`	Return 0 to the OS = success

2. Data Types

C has a small set of primitive types. Unlike higher-level languages, their exact size can vary by platform — always use sizeof() when size matters.

2.1 Primitive Types

Type	Description
`int`	Signed integer, typically 32-bit. e.g. int x = 42;
`long`	Larger integer, at least 32-bit, often 64-bit
`long long`	At least 64-bit integer
`short`	Smaller integer, at least 16-bit
`char`	Single byte — used for characters and small integers
`float`	32-bit floating point (less precision)
`double`	64-bit floating point — preferred for decimals
`long double`	Extended precision float (80 or 128 bit)
`void`	Represents no type — used for functions that return nothing
`_Bool / bool`	Boolean: 0 = false, any other value = true (include <stdbool.h>)

2.2 Type Modifiers

Modified Type	Description
`unsigned int`	Only positive values, range doubles (0 to ~4B on 32-bit)
`signed char`	Explicitly signed byte (-128 to 127)

Modified Type	Description
`unsigned char`	Byte as 0–255, common for binary data
`const int x = 5;`	Value cannot be changed after initialization

2.3 Fixed-Width Types (Preferred in Modern C)

Include <stdint.h> for platform-independent sizes:

```
#include <stdint.h>


int8_t   a = -100;     // exactly 8 bits, signed
uint8_t  b = 200;      // exactly 8 bits, unsigned
int16_t  c = 30000;
uint32_t d = 4000000000U;
int64_t  e = -9000000000000LL;
```

2.4 sizeof Operator

```
printf("%zu\n", sizeof(int));      // typically 4
printf("%zu\n", sizeof(double));   // typically 8
printf("%zu\n", sizeof(char));     // always 1
```

3. Variables & Declarations

3.1 Declaring and Initializing

```
int age;              // declaration only (value is garbage!)
int age = 25;         // declaration + initialization
double pi = 3.14159;
char letter = 'A';
char name[] = "Alice"; // string (char array)

// Multiple declarations
int x = 1, y = 2, z = 3;

// const — value cannot change
const double TAX_RATE = 0.08;
```

3.2 Scope Rules

```
int global = 10;   // global scope — visible everywhere

void myFunc(void) {
    int local = 20; // local scope — only inside this function
    {
        int block = 30; // block scope — only inside these braces
    }
    // block is NOT accessible here
}
```

3.3 Storage Classes

Keyword	Behavior
`auto`	Default for local variables — lives on the stack
`static (local)`	Persists between function calls — initialized once
`static (global)`	Limits visibility to the current file (file-private)

Keyword	Behavior
`extern`	Declares a variable defined in another file
`register`	Hint to compiler to keep in CPU register (rarely used)

4. Operators

4.1 Arithmetic Operators

Operator	Description
`x + y`	Addition
`x - y`	Subtraction
`x * y`	Multiplication
`x / y`	Division — integer division truncates: 7/2 = 3
`x % y`	Modulo (remainder): 7 % 2 = 1
`x++` / `x--`	Post-increment/decrement (returns old value, then changes)
`++x` / `--x`	Pre-increment/decrement (changes, then returns new value)

4.2 Comparison & Logical Operators

Operator	Description
`==` `!=`	Equal, Not equal — NOTE: = is assignment, == is comparison!
`<` `>` `<=` `>=`	Less than, greater than, etc.
`&&`	Logical AND — short-circuits (stops at first false)
`||`	Logical OR — short-circuits (stops at first true)
`!`	Logical NOT

4.3 Bitwise Operators

```
int a = 0b1010;  // 10 in binary
int b = 0b1100;  // 12 in binary


a & b   // AND:  0b1000 = 8
a | b   // OR:   0b1110 = 14
a ^ b   // XOR:  0b0110 = 6
~a      // NOT:  flips all bits
a << 2  // Left shift:  multiply by 4
a >> 1  // Right shift: divide by 2
```

4.4 Assignment Operators

Operator	Equivalent To
`x = 5`	Basic assignment
`x += 3`	x = x + 3
`x -= 3`	x = x - 3
`x *= 2`	x = x * 2
`x /= 2`	x = x / 2
`x %= 3`	x = x % 3
`x &= mask`	Bitwise AND-assign
`x |= flag`	Bitwise OR-assign (set bits)
`x ^= toggle`	Bitwise XOR-assign (flip bits)
`x <<= 1`	Left shift assign
`x >>= 1`	Right shift assign

4.5 The Ternary & Comma Operators

```
// Ternary: condition ? value_if_true : value_if_false
int max = (a > b) ? a : b;


// Comma operator: evaluates left, discards, then evaluates right
int x = (a++, b++, a + b); // a and b are incremented, x = their sum
```

5. Control Flow

5.1 if / else if / else

```
if (x > 0) {
    printf("positive\n");
} else if (x < 0) {
    printf("negative\n");
} else {
    printf("zero\n");
}
```

5.2 switch Statement

```
switch (day) {
    case 1:  printf("Monday\n");  break;
    case 2:  printf("Tuesday\n"); break;
    // ...
    default: printf("Weekend\n"); break;
}
// NOTE: Without 'break', execution FALLS THROUGH to the next case!
```

5.3 Loops

for Loop

```
for (int i = 0; i < 10; i++) {
    printf("%d ", i);  // prints 0 through 9
}
// Syntax: for (init; condition; update) { body }
```

while Loop

```
int n = 0;
while (n < 5) {
    printf("%d\n", n);
    n++;
}
```

do-while Loop (runs at least once)

```
int input;
do {
    printf("Enter a positive number: ");
    scanf("%d", &input);
} while (input <= 0);
```

5.4 break, continue, goto

Keyword	Description
`break`	Exit the nearest enclosing loop or switch immediately
`continue`	Skip to the next iteration of the loop
`goto label;`	Jump to a label — use sparingly, mainly for error cleanup

6. Functions

6.1 Defining and Calling Functions

```
// Syntax: return_type function_name(param_type param_name, ...) { ... }


int add(int a, int b) {
    return a + b;
}


void greet(char *name) {          // void = no return value
    printf("Hello, %s!\n", name);
}


int main(void) {
    int result = add(3, 4);       // result = 7
    greet("Alice");
    return 0;
}
```

6.2 Function Prototypes (Declarations)

In C, functions must be declared before they are used. A prototype tells the compiler the signature without defining the body.

```c
// Prototype at top of file (or in a .h header)
int add(int a, int b);
void greet(char *name);

int main(void) {
    add(2, 3);  // OK — prototype seen above
    return 0;
}

int add(int a, int b) { return a + b; } // definition later
```

6.3 Pass by Value vs. Pass by Pointer

C is strictly pass-by-value. To modify a variable in the caller, pass its address (pointer).

```c
// Pass by value — caller's x is NOT modified
void doubleVal(int x) { x *= 2; }

// Pass by pointer — caller's x IS modified
void doublePtr(int *x) { *x *= 2; }

int n = 5;
doubleVal(n);   // n is still 5
doublePtr(&n);  // n is now 10  — & gets address, * dereferences
```

6.4 Variadic Functions

```c
#include <stdarg.h>


double average(int count, ...) {
    va_list args;
    va_start(args, count);
    double sum = 0;
    for (int i = 0; i < count; i++)
        sum += va_arg(args, double);
    va_end(args);
    return sum / count;
}
// average(3, 1.0, 2.0, 3.0) == 2.0
```

6.5 Recursive Functions

```c
int factorial(int n) {
    if (n <= 1) return 1;       // base case
    return n * factorial(n - 1); // recursive case
}
```

7. Pointers

Pointers are one of C's most powerful — and most misunderstood — features. A pointer is a variable that stores the memory address of another variable.

7.1 Pointer Basics

```
int x = 42;
int *p = &x;      // p holds the address of x
                  // & = address-of operator

printf("%d\n", *p);  // 42 — * dereferences: reads the value AT the address
*p = 100;             // modifies x through the pointer; x is now 100

printf("%p\n", (void*)p); // prints the address itself, e.g. 0x7ffee2c
```

7.2 Pointer Arithmetic

```
int arr[] = {10, 20, 30, 40};
int *p = arr;          // p points to arr[0]

printf("%d\n", *p);       // 10
p++;                       // advance by sizeof(int) bytes
printf("%d\n", *p);       // 20
printf("%d\n", *(p + 2)); // 40 — two elements ahead

// p[n] is exactly equivalent to *(p + n)
```

7.3 Pointers and Arrays

Array names decay to a pointer to the first element. Pointers and arrays are deeply intertwined in C.

```
char str[] = "hello";
char *p = str;       // same as p = &str[0]

while (*p != '\0') { // iterate until null terminator
    putchar(*p);
    p++;
}
```

7.4 Pointer to Pointer

```
int x = 5;
int *p = &x;
int **pp = &p;   // pointer to a pointer

printf("%d\n", **pp);  // 5 — double dereference
```

7.5 NULL Pointers & Safety

```
#include <stdio.h>

int *p = NULL;   // safe initialization — pointer to nothing

if (p != NULL) { // always check before dereferencing!
    printf("%d\n", *p);
}
// Dereferencing NULL causes a segmentation fault (crash)
```

7.6 Function Pointers

```
// Declare: return_type (*name)(param_types)
int (*operation)(int, int);

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

operation = add;
printf("%d\n", operation(3, 4));  // 7
operation = mul;
printf("%d\n", operation(3, 4));  // 12

// Common use: callbacks and dispatch tables
```

8. Arrays & Strings

8.1 Arrays

```
// Declare: type name[size];
int scores[5];                          // uninitialized — values are garbage
int scores[5] = {90, 85, 78, 92, 88}; // initialized
int scores[] = {90, 85, 78};            // size inferred (3 elements)
int matrix[3][4];                       // 2D array: 3 rows, 4 columns

// Access: zero-indexed
scores[0] = 100;   // first element
scores[4] = 75;    // last element (index = size - 1)

// Iterate
for (int i = 0; i < 5; i++) {
    printf("%d ", scores[i]);
}
```

⚠ **Warning:** **C does NOT bounds-check arrays. Accessing scores[10] on a 5-element array is undefined behavior — it may crash or silently corrupt memory.**

8.2 Strings (char arrays)

C strings are arrays of char terminated by a null character '\0'. There is no built-in String type.

```c
#include <string.h>


char name[20] = "Alice";    // stored as: A l i c e \0 ...
char *greeting = "Hello";   // string literal — read-only!


// String functions from <string.h>:
strlen(name)            // length (not counting \0): 5
strcpy(dest, src)       // copy src into dest (dest must be large enough!)
strncpy(dest, src, n)   // safer: copy at most n chars
strcat(dest, src)       // append src to dest
strcmp(a, b)            // 0 if equal, <0 if a<b, >0 if a>b
strncmp(a, b, n)        // compare first n characters
strchr(str, 'c')        // pointer to first occurrence of 'c', or NULL
strstr(hay, "needle")   // pointer to first substring, or NULL
sprintf(buf, fmt, ...)  // format string into buffer
snprintf(buf, n, ...)   // safer: write at most n bytes
```

9. Structs, Unions & Enums

9.1 struct — Grouping Related Data

```
// Define the struct type
struct Point {
    double x;
    double y;
};

// Use typedef for cleaner syntax (very common in C code)
typedef struct {
    char name[50];
    int age;
    double salary;
} Employee;

// Create and use instances
Employee emp = {"Alice", 30, 75000.0};
printf("%s is %d\n", emp.name, emp.age);

// Access via pointer uses -> operator
Employee *ptr = &emp;
printf("%s\n", ptr->name);   // equivalent to (*ptr).name
```

9.2 union — Shared Memory

A union allocates only enough memory for its largest member. All members share the same address. Only one member is valid at a time.

```
union Data {
    int   i;
    float f;
    char  str[4];
};

union Data d;
d.i = 42;          // write as int
printf("%f\n", d.f); // reading as float — not meaningful, but valid syntax
// sizeof(union Data) == sizeof(largest member) == 4
```

9.3 enum — Named Integer Constants

```
typedef enum {
    NORTH = 0,
    SOUTH,    // 1
    EAST,     // 2
    WEST      // 3
} Direction;

Direction dir = NORTH;
if (dir == NORTH) printf("Going north!\n");
```

10. Dynamic Memory Management

Unlike higher-level languages, C requires you to manually request and release memory from the heap. The relevant functions are in <stdlib.h>.

Function	Description
`malloc(size)`	Allocate size bytes — contents are uninitialized (garbage)
`calloc(n, size)`	Allocate n * size bytes — zeroed out
`realloc(ptr, size)`	Resize a previously allocated block
`free(ptr)`	Release memory back to the OS. MUST be called for every malloc/calloc

10.1 Basic malloc / free

```
#include <stdlib.h>


int *arr = malloc(10 * sizeof(int));   // allocate array of 10 ints


if (arr == NULL) {                      // ALWAYS check for NULL!
    fprintf(stderr, "Out of memory!\n");
    return 1;
}


for (int i = 0; i < 10; i++) arr[i] = i * i;


free(arr);     // release the memory
arr = NULL;    // good practice: prevents use-after-free bugs
```

10.2 Dynamic Struct Allocation

```
typedef struct { int x; int y; } Point;


Point *p = malloc(sizeof(Point));
p->x = 10;
p->y = 20;
free(p);
```

10.3 Common Memory Bugs

Bug	Description
Memory leak	malloc without a corresponding free
Use-after-free	Accessing memory after calling free()
Double free	Calling free() twice on the same pointer — crash/corruption
Buffer overflow	Writing past the end of an allocated block
Dangling pointer	Pointer to memory that has been freed

Tool tip: Use Valgrind (Linux) or AddressSanitizer (compile with -fsanitize=address) to detect memory errors.

11. File I/O

```c
#include <stdio.h>

// Opening a file
FILE *fp = fopen("data.txt", "r");  // modes: r, w, a, r+, w+, rb, wb
if (fp == NULL) { perror("fopen"); return 1; }

// Reading line by line
char line[256];
while (fgets(line, sizeof(line), fp) != NULL) {
    printf("%s", line);
}

fclose(fp);  // ALWAYS close the file

// Writing to a file
FILE *out = fopen("output.txt", "w");
fprintf(out, "Value: %d\n", 42);
fclose(out);

// Binary I/O
fread(buffer, sizeof(int), count, fp);   // read count ints
fwrite(buffer, sizeof(int), count, fp);  // write count ints

// File positioning
fseek(fp, 0, SEEK_SET);   // go to start
fseek(fp, 0, SEEK_END);   // go to end
long pos = ftell(fp);     // get current position
rewind(fp);               // equivalent to fseek(fp, 0, SEEK_SET)
```

Mode	Description
`"r"`	Read — file must exist
`"w"`	Write — creates or truncates
`"a"`	Append — creates or appends to existing
`"r+"`	Read and write — file must exist
`"w+"`	Read and write — creates or truncates
`"rb"` / `"wb"`	Binary read/write mode

12. The C Preprocessor

Before compilation, the preprocessor handles directives starting with #. These are text-substitution operations, not C code.

12.1 Common Directives

Directive	Purpose
`#include <stdio.h>`	Include a system header
`#include "myfile.h"`	Include a local header file
`#define PI 3.14159`	Define a constant macro
`#define MAX(a,b) ((a)>(b)?(a):(b))`	Function-like macro
`#undef PI`	Remove a macro definition
`#ifdef` / `#ifndef`	Compile block only if macro is/isn't defined
`#if` / `#elif` / `#else` / `#endif`	Conditional compilation with expressions
`#pragma once`	Include guard — prevent double-inclusion of a header
`#error "message"`	Emit a compile-time error

12.2 Include Guards (Header Files)

```
// mymath.h
#ifndef MYMATH_H    // if MYMATH_H is not yet defined...
#define MYMATH_H    // ...define it (so this block runs only once)

int add(int a, int b);
double sqrt_approx(double x);

#endif  // MYMATH_H
```

13. printf / scanf Format Specifiers

Specifier	Prints
`%d` or `%i`	Signed decimal integer (int)
`%u`	Unsigned decimal integer
`%ld / %lld`	long / long long
`%f`	Float or double (6 decimal places by default)
`%.2f`	Float with exactly 2 decimal places
`%e`	Scientific notation: 3.140000e+00
`%g`	Shorter of %f or %e
`%c`	Single character
`%s`	C string (null-terminated char array)
`%p`	Pointer address in hex
`%x / %X`	Unsigned hex (lowercase / uppercase)
`%o`	Unsigned octal
`%zu`	size_t (result of sizeof, array lengths)
`%%`	Literal % character

Width & Padding

```
printf("%10d", 42);     // right-aligned in field of width 10: '        42'
printf("%-10d", 42);    // left-aligned:                       '42        '
printf("%010d", 42);    // zero-padded:                        '0000000042'
printf("%+d", 42);      // always show sign:                   '+42'
```

14. Common Standard Library Headers

Header	Key Functions / Types
<stdio.h>	printf, scanf, fopen, fclose, fgets, fprintf, sprintf
<stdlib.h>	malloc, free, exit, atoi, atof, rand, qsort, bsearch
<string.h>	strlen, strcpy, strcmp, strcat, memcpy, memset
<math.h>	sqrt, pow, sin, cos, fabs, floor, ceil, log (link with -lm)
<ctype.h>	isdigit, isalpha, isupper, tolower, toupper
<stdint.h>	int8_t, uint32_t, int64_t and other fixed-width types
<stdbool.h>	bool, true, false (C99+)
<stddef.h>	NULL, size_t, ptrdiff_t, offsetof
<errno.h>	errno — error number set by library calls
<time.h>	time, clock, difftime, struct tm, strftime
<assert.h>	assert(expr) — aborts if expr is false (debug aid)
<limits.h>	INT_MAX, INT_MIN, CHAR_MAX, etc.

15. Compilation, Flags & Best Practices

15.1 GCC / Clang Compilation Flags

Flag	Purpose
`-o output`	Name the output executable
`-Wall -Wextra`	Enable all common warnings — use always!
`-Werror`	Treat warnings as errors
`-g`	Include debug symbols for gdb
`-O2 / -O3`	Optimization level (0=none, 1=basic, 2=good, 3=aggressive)
`-std=c11`	Compile as C11 standard (also: c89, c99, c17, c23)
`-lm`	Link the math library (needed for <math.h>)
`-fsanitize=address`	Enable AddressSanitizer for memory error detection
`-fsanitize=undefined`	Enable UBSan — catch undefined behavior at runtime

15.2 Key Best Practices

- Always initialize variables — uninitialized values are garbage in C
- Check return values — fopen, malloc, scanf all return NULL/errors
- Use const for read-only data and function parameters that shouldn't change
- Prefer snprintf over sprintf to avoid buffer overflows
- Always match every malloc with exactly one free
- Use -Wall -Wextra during development — fix every warning
- Use size_t for sizes and array indices (it's unsigned and the right width)
- Use typedef for struct and enum types to avoid writing struct everywhere
- Put declarations in .h headers, definitions in .c source files
- Never return a pointer to a local variable — it will be destroyed on return

15.3 C Standards Quick Reference

Standard	Notable Features
C89 / ANSI C	Original standard — very portable, no // comments, no mixed declarations
C99	Added: bool, // comments, variable-length arrays, inline, <stdint.h>
C11	Added: _Generic, threads, atomics, anonymous structs/unions, _Static_assert
C17 / C18	Bug-fix release for C11 — no major new features

Standard	Notable Features
C23	Added: typeof, nullptr, [[attributes]], binary literals (0b1010)

16. C vs. Other Languages — Quick Comparison

Feature	C's Approach
String	No built-in string type — use char[] + <string.h>
Boolean	No bool in C89 — use int or #include <stdbool.h> in C99+
Classes/Objects	No OOP — use structs + function pointers to simulate
Exceptions	No exceptions — use return codes, errno, or setjmp/longjmp
Garbage collection	No GC — you manage all memory manually with malloc/free
Generics/templates	No generics — use void* pointers or macros (with care)
Namespaces	No namespaces — prefix your function names (e.g., vec_push)
Array length	No .length — track it yourself or use sizeof(arr)/sizeof(arr[0])
For-each	No range-for — use a standard for loop with an index
Import/module	No modules — use #include for headers and link .o files

You now have a complete reference to C.

The best next step: write code. Start with small programs, use -Wall, and read the errors.